Universidad Pascual Bravo

Facultad de Ingeniería

Programa: Tecnología en Desarrollo de Software

Asignatura: Bases de datos I G(101)

Docente: Oralia Cortes Grajales

S1. Tarea. Comparar la implementación de un diccionario de datos físico en diferentes SGBD

Estudiante: Juan Esteban Villada Ruiz

C.C 1039049872

Medellín – Colombia

Octubre 25 de 2026

INFORME TÉCNICO COMPARATIVO E IMPLEMENTACIÓN MULTI-SGBD

PostgreSQL | MySQL | SQL Server

1. Introducción y Justificación de la Estrategia Multi-SGBD

El presente informe documenta la transición del modelo lógico al modelo físico de la base de datos **Red_Social_PascualinaV2**, diseñada para gestionar la interacción académica, mentorías, eventos y recursos dentro de la comunidad estudiantil de la Institución Universitaria Pascual Bravo.

Con el fin de garantizar una arquitectura robusta, portátil e interoperable, el diseño físico se desplegó y validó de forma nativa en tres motores de bases de datos líderes en la industria: **PostgreSQL (v15+)**, **Microsoft SQL Server (2019/2022)** y **MySQL (8.0+)**. Cada SGBD aporta capacidades operativas únicas que responden a distintos escenarios empresariales:

- **PostgreSQL:** Ideal para analítica avanzada, manejo eficiente de tipos de datos JSONB binarios y extensiones espaciales/IoT.
- **Microsoft SQL Server:** Excelente para entornos corporativos Microsoft, integración con Azure y rendimiento transaccional de alta escala.
- **MySQL:** Alternativa liviana de alta disponibilidad para servicios web, fácil escalabilidad horizontal y despliegues en la nube.

2. Paso a Paso de la Implementación Física y DDL

2.1. Orden de Creación de Tablas y Limpieza Preventiva

Para prevenir errores de dependencia circular o violaciones de integridad referencial durante el despliegue de los scripts DDL, se estableció un orden estricto de eliminación (DROP TABLE IF EXISTS en orden inverso) y creación (CREATE TABLE en orden jerárquico directriz):

1. **Tablas Independientes (Padres):** estudiante, interes, grupo_estudio, evento.
2. **Tablas Intermedias (Hijas y Asociativas):** publicacion, equipo, estudiante_interes, integrante_grupo, recurso_academico, notificacion.
3. **Tablas Dependientes (Nivel 2/3):** comentario, reaccion, miembro_equipo, mentoria, mensaje_directo.

2.2. Manejo de Integridad Referencial y Múltiples Caminos en Cascada (*Cascade Paths*)

Durante la fase de pruebas físicas en Microsoft SQL Server Management Studio (SSMS), se identificó un conflicto técnico derivado de las reglas de integridad referencial en las tablas mentoria, mensaje_directo, comentario y reaccion. SQL Server prohíbe la definición de múltiples Foreign Keys con ON DELETE CASCADE que apunten a la misma tabla abuela (estudiante) a través de rutas directas e indirectas (Error Msg 1785).

Para resolver esta restricción técnica en SQL Server, se ajustaron las claves foráneas de id_publicacion y id_receptor/id_mentor a ON DELETE NO ACTION ON UPDATE NO ACTION, manteniendo la integridad de los datos sin comprometer la ejecución del script DDL. En PostgreSQL y MySQL, el motor admite ON DELETE CASCADE directo sin bloqueos de compilación.

3. Cuadro Comparativo Técnico entre SGBD

Característica / Atributo	**PostgreSQL**	**MySQL**	**SQL Server**
Autonumérico (PK)	GENERATED ALWAYS AS IDENTITY	AUTO_INCREMENT	IDENTITY(1,1)
Cadenas de Texto	VARCHAR(N) / TEXT	VARCHAR(N) / TEXT	NVARCHAR(N) / NVARCHAR(MAX)
Fechas y Tiempos	TIMESTAMP / DATE	DATETIME / DATE	DATETIME2 / DATE
Valores Booleanos	BOOLEAN (true/false)	TINYINT(1) (0/1)	BIT (0/1)
Soporte NoSQL / JSON	JSONB (Binario e Indizable)	JSON (Validación Nativa)	NVARCHAR(MAX) + CHECK ISJSON
Reglas de Cascada (FK)	Soporte Cascadas Múltiples	Soporte Cascadas Múltiples	Restricción No Action en Rutas Múltiples

4. Justificación del Almacenamiento Híbrido (SQL + NoSQL/JSON)

En el desarrollo de plataformas modernas de redes sociales e IoT, la rigidez del modelo relacional estricto puede convertirse en un cuello de botella para datos de estructura altamente variable, no estructurada o de rápido crecimiento. Para mitigar esta limitante sin renunciar a las garantías ACID de las bases de datos relacionales, se implementó un modelo de almacenamiento híbrido incorporando columnas JSON/JSONB en 3 tablas estratégicas:

1. **Tabla publicacion (campo metadatos_iot):** Permite registrar datos de telemetría de dispositivos móviles (coordenadas GPS, modelo de teléfono, dirección IP, red Wi-Fi o datos de acelerómetro) al momento de realizar una publicación. Al no requerir campos fijos, la estructura puede evolucionar dinámicamente sin modificar el esquema DDL de la base de datos.
2. **Tabla evento (campo detalles_evento):** Diseñado para almacenar lecturas de sensores de aforo de recintos, nivel de ruido, temperatura ambiental o equipamiento técnico requerido para actividades del campus. Estos datos varían dramáticamente dependiendo de si el evento es presencial, virtual o híbrido.
3. **Tabla notificacion (campo payload_telemetria):** Funciona como un almacén de Big Data / Logs para registrar el payload completo de auditoría, trazabilidad de

dispositivos receptores, códigos de respuesta de servidores push y métricas de latencia.

Ventajas Técnicas del Enfoque Híbrido: PostgreSQL procesa los campos JSONB mediante un formato binario optimizado que permite la creación de índices GIN (*Generalized Inverted Index*), habilitando búsquedas hiperrápidas dentro del documento JSON. Por su parte, MySQL y SQL Server aseguran la validez estructural de los documentos mediante funciones como ISJSON().

5. Conclusiones y Propuestas de Mejora

- **Éxito del Despliegue Multi-SGBD:** Se validó la viabilidad técnica y compatibilidad de las 15 tablas del esquema Red_Social_PascualinaV2, demostrando que el modelo lógico puede adaptarse a las particularidades de PostgreSQL, SQL Server y MySQL.
- **Arquitectura Flexible y Escalable:** La inclusión de los campos JSONB/JSON dota a la plataforma de capacidades para Big Data e Internet de las Cosas (IoT), posicionándola como una solución preparada para el análisis masivo de datos estudiantiles.
- **Propuesta de Mejora (Indexación Especial/GIN):** Se sugiere crear índices GIN sobre metadatos_iot en PostgreSQL y particionamiento horizontal por rango de fechas en la tabla notificacion para mantener la eficiencia en entornos de producción con millones de registros.